

Payroll

Status and Way Forward

Why June's bank transfer could not be posted, what has been fixed, and the wider finding it uncovered — that a posted ledger in this system can be deleted by anyone with a Reset button, and that no accounting period has ever been closed.

DATE 6 September 2026	BUILD master @ b6035a96	BLOCKED Payment 16855 — 322,078	LEDGER EXPOSED 5,065 posted entries
--------------------------	----------------------------	---------------------------------------	--

Contents

Part I — The payroll incident

- 1 Summary
- 2 What failed, and why
- 3 What has been refactored
- 4 Clearing payment 16855
- 5 Payroll issue register

Part II — The wider finding

- 6 Reset and delete
- 7 Period close
- 8 The editing policy

Part III

- 9 Way forward
- 10 How these figures were verified

PART I

The payroll incident

1 Summary

June's bank salary transfer — payment **16855, 322,078** — cannot be moved to `PMNT_SENT`. It fails with a message about a missing accrued account on party `276`, which is misleading: party 276 is `موظفي الشركة`, the shared staff party the batch addresses its two settlement payments to. It is not a person and has no salary account by design.

The invoice edit that appeared to trigger this was only the last of **three** changes that moved out from under a payment created seven weeks earlier. The immediate cause is that an aggregate payroll payment records no link to the invoices it settles, so the posting code re-derives that set from live master data every time — and any later edit silently changes the answer.

Two fixes are in master (commit `130429d1`) and need no migration. They stop employee-preference edits from re-partitioning historical runs, and they stop a run payment from being misclassified as an individual one. Together they remove this failure mode for every future run. A reviewed SQL script now exists to clear the June payment itself — section 4.

Investigating this surfaced something larger. The user was able to edit a June invoice in late August — after the payments were sent and after July's payroll had already been generated — because the system offers a **Reset** action that deletes posted accounting transactions outright. Thirteen places in the codebase do this. **5,065 of the ledger's 5,084 transactions are posted and reachable this way**, and no accounting period has ever been closed. This is Part II, and it is the more important half of this document.

2 What failed, and why

2.1 Three changes, seven weeks apart

When the batch ran on 5 July it produced two payments, each equal to the sum of the invoices it covered. Both were correct at that moment.

When	What happened	Effect on the June bank run
5 Jul 09:41	Batch created 50 June payroll invoices and both settlement payments. Party 155 (60,000, احمد عبدالرحمن) was <code>BANK_TRANSFER</code> at that moment, so he was part of the bank group.	19 employees 262,078 + party 155 60,000 = 322,078 ✓ matched
19 Jul	Cash payment 16849 sent and posted — 330,684 across 30 employees. Correct, and unaffected by everything below.	—
5 Aug 12:21	Party 155 was edited and his payroll method changed to <code>CASH</code> . This silently removed him from a bank run that had already been executed and paid for.	322,078 → 262,078
25 Aug 07:18	INV1038 (احمد عبد العال, party 169) was reverted to <code>INVOICE_IN_PROCESS</code> , its 2-day absence deduction of 667 removed, and set back to <code>INVOICE_READY</code> . Its original accounting transaction was deleted and a new one created at the full 10,000.	262,078 → 262,745

By the time the payment was sent, it was being compared against a run total that had moved twice — once by 60,000 and once by 667 — for a net difference of **59,333**.

2.2 Why the error named party 276

The posting code decided *what kind of payment this is* by testing whether its amount still equalled the run total:

```
bool isAggregateRunSettlement =  
    string.IsNullOrEmpty(payment.OverrideGlAccountId) && runTotal == paymentAmount;
```

That is a classification derived from arithmetic that is recomputed on every attempt. When the totals stopped agreeing, a whole-run payment was reclassified as a payment to one individual — so the code went looking for an employee salary account on party 276 and, correctly, did not find one. The message was accurate about the lookup and useless about the cause.

3 What has been refactored

Three changes, all in commit `130429d1`. No database migration: the snapshot uses the existing `INVOICE_ATTRIBUTE` table, which was empty and is already cleaned up by both payroll delete paths.

3.1 Each invoice records the method it was run under

`BatchCreatePayrollInvoices` now writes an `InvoiceAttribute` (`PAYROLL_PMT_METHOD`) on every payroll invoice as it is created, and `GeneralLedgerService` reads that snapshot when deciding which invoices a payment covers — instead of reading the employee's *current* preference.

What it serves: a run becomes a historical fact. Changing an employee's payment method affects the next run and nothing before it. The 5 August edit that removed party 155 from a June run retroactively could not happen again.

The value is stored verbatim rather than normalised, and the fallback for older invoices keys on the *absence of the attribute row* rather than on a null value. Both details matter: an employee with no preference set belongs to neither payment, and must stay excluded rather than defaulting into one and unbalancing the transaction.

3.2 A payment's identity comes from its recipient, not its arithmetic

```
bool isAggregateRunSettlement =
    payment.PartyIdTo == PayrollConstants.StaffPartyId
    && string.IsNullOrEmpty(payment.OverrideGlAccountId);
```

The `runTotal == paymentAmount` comparison survives, but demoted from a classification to a *validation* that reports both figures and the difference. **What it serves:** a payment can no longer change what it *is* because a number moved. This is also the marker `GetProjectPayroll`, `GetCompanyReport.GetPayroll` and `Fact_Project_Payroll` already use, so the GL posting and the reports now agree on the definition rather than each inferring it separately. The same situation now reports itself:

```
Payroll payment 16855 settles the whole BANK_TRANSFER run for 06/2026, but its amount
no longer matches that run: payment 322078.00 vs run total 262745.00 across 19
invoice(s), a difference of 59333.00. The run has changed since the payment was
created (an invoice was edited, or an employee's payroll payment method was changed).
Correct the payment amount or the invoices so the two agree, then post again.
```

3.3 Named constants and two sharper failures

A new `PayrollConstants` class replaces the literals that had to agree across four files but were enforced nowhere. Two further failures were upgraded from silent or vague to specific: an employee missing an `ACCOUNTS_PAYABLE` account used to log a warning, skip their debit, and then fail the balance guard with bare arithmetic — it now names the employee and invoice; and the ad-hoc missing-account error now explains what an ad-hoc payroll payment is expected to look like.

Do not backfill the snapshot from current preferences. For June that would write `CASH` for party 155 and bake in the exact defect being fixed. The fallback handles historical invoices correctly; set attributes by hand only where the true run membership is known.

4 Clearing payment 16855

One payment is doing two jobs: settling nineteen bank employees *and* paying one cash employee. The correction splits it, and restores the deduction at its origin rather than papering over it with a journal entry.

Scripted as `API/Sql/Production/fix_june_payroll_inv1038_2026_09_06.sql` — guarded, idempotent, and dry-run against a copy of the database inside a transaction that was rolled back.

Action	Detail
1 Restore the 667 on INV1038 (SQL)	Insert the <code>PAYROL_DD_ABSENCE</code> item at quantity 2 , amount 333.50 — the batch's days × per-day-rate convention, so the absence reports still read 2 days.
2 Correct the accrual (SQL)	Transaction 20788, both legs 10,000 → 9,333 (D 601000 / C 220013).
3 Set payment 16855 to 262,078 (SQL)	Still <code>PMNT_NOT_PAID</code> , so the amount is editable and no posting exists behind it.
4 Create party 155's own payment (UI)	<code>PAYROL_PAYMENT</code> , to party 155 , <code>CIB_CHECKING</code> , 60,000, effective 2026-06-30. Posts against his accrued account <code>220008</code> , unpaid since June. Left to the UI because a payment needs a sequenced id and drives <code>FinAccountTrans</code> through the application.
5 Send both (UI)	16855 posts as the run settlement (262,078 = 262,078); the 60,000 posts as an individual payment.

	Now	After
INV1038 (احمد عبد العال)	10,000	9,333
Bank run — 19 employees	262,745	262,078
Payment 16855	322,078 ✗	262,078 ✓
Payment to party 155	—	60,000 ✓
Manual journal entry required	667	none

The script restores what the batch originally produced on 5 July rather than inventing a correction, so no reversing entry is needed. Three independent facts confirm the original figure: the payment's own comment (*667 تعديل خصم 2 يوم احمد عبد العال*), the arithmetic ($262,078 + 667 = 262,745$), and 2 days × 10,000 ÷ 30 rounding to 667. INV1038 is the only June payroll invoice that was touched after the batch created it, and no July invoice was touched.

Do not re-run June payroll after this. `BatchCreatePayrollInvoices` deletes and rebuilds every invoice and every payment addressed to party 276 for the month, which would undo the correction — and it deletes payments to 276 *only*, so the new payment to party 155 would survive and then double-count.

Side effect worth knowing

If step 1 is ever done through the payroll item editor rather than this script, that screen saves absence as `quantity 1`, `amount 667` while the batch saves `quantity 2`, `amount 333.50`. The money is identical, but the absence report reads `quantity` as days, so the employee would show 1 day instead of 2. The script writes the batch's shape. This is issue 6 in section 5.

5 Payroll issue register

Found during a full read of the payroll path — backend, GL posting, frontend, seed data — and verified against the database.

#	Issue	Detail	Status
1	No recorded run membership	An aggregate payment records no link to the invoices it settles — no <code>PaymentApplication</code> , no payment group. The set is re-derived on every posting attempt. Section 3.1 mitigates it; it does not remove it.	HIGH
2	Two competing deduction engines	<code>BatchCreatePayrollInvoices</code> marks advances deducted, then <code>SetInvoiceStatus(READY)</code> fires <code>ApplyPayrollDeductionsToAdvances</code> , which independently sweeps every approved advance dated in the previous month and marks it fully paid — including advances the run never deducted. Different rules, different status constants, same invoice.	HIGH
3	Invalid schedule status values	The batch writes <code>"PAID"</code> , <code>ApplyPayrollDeductionsToAdvances</code> writes <code>"SCHD_PAID"</code> , reverts write <code>"SCHEDULED"</code> . <code>STATUS_ITEM</code> defines <code>SCHED_PENDING</code> / <code>SCHED_DUE</code> / <code>SCHED_PAID</code> . The table holds 45 <code>SCHEDULED</code> and 26 <code>PAID</code> — neither is a valid status id.	MEDIUM
4	Payroll item types missing from seed	<code>PAYROL_OVERTIME</code> , <code>PAYROL_DD_ABSENCE</code> and <code>PAYROL_DD_ADVANCE</code> exist only in the live database, not in <code>invoice_item_types.json</code> . A fresh environment seeds a payroll module that cannot run.	MEDIUM
5	Attendance computed only in the browser	The fingerprint Excel is parsed entirely client-side. Nothing server-side records the file or how absence days were derived, so a figure cannot be re-audited after the fact.	MEDIUM
6	Absence days lost on manual edit	The payroll item editor saves <code>quantity 1</code> with the full amount; the batch saves <code>quantity = days</code> with the per-day rate. Money agrees, day counts do not — and both absence and payroll reports read <code>quantity</code> as days.	MEDIUM
7	Hardcoded values	Partly addressed by <code>PayrollConstants</code> . Remaining: <code>CIB_CHECKING</code> , <code>EGP</code> , <code>601000</code> , the 30-day month, and the Friday/Saturday weekend.	LOW
8	<code>CheckExistingPayrollInvoices</code> flaws	Accepts <code>OrganizationPartyId</code> but never applies it, and its month end is <code>monthStart.AddMonths(1)</code> — so the 1st of the next month counts as the current month.	LOW
9	Dead OFBiz heritage	<code>PayrollPreference</code> , <code>SalaryStepNew</code> and <code>PayGrade</code> are mapped but never queried. <code>PayrollReport.tsx</code> is unreferenced; <code>Temps/ListPayrollData.cs</code> is a superseded v1.	LOW
10	Column naming	<code>EMPLOYEE_ADVANCE.AdvanceTypeId</code> is an EF-defaulted <code>longtext</code> , unlike every other snake-case column in the schema.	LOW

6 Reset and delete

The reason a June invoice could be edited in late August — after the payments were sent, and after July's payroll had been generated — is that the system offers a **Reset** action which returns an artifact to draft status and **physically deletes its posted accounting transactions**. The user was not circumventing a control. He pressed a button the system provides, and it worked.

6.1 The scale

13 code paths delete posted ledger rows.

Handler	Deletes
ResetInvoice	Status → <code>INVOICE_IN_PROCESS</code> , then four <code>ExecuteDeleteAsync</code> calls including <code>AcctgTrans</code> and its entries
ResetPayment	Status → <code>PMNT_NOT_PAID</code> , deletes <code>PaymentApplication</code> , <code>AcctgTransEntry</code> , <code>AcctgTrans</code>
ResetProjectCertificate ResetMultiPaymentCertificate	Same pattern for construction certificates
DeletePayment · DeleteAcctgTrans DeleteSalesRequest DeleteMultiPaymentCertificate DeleteEmployeeAdvance	Physical deletion of the artifact and its ledger rows
DeletePayrollInvoices BatchCreatePayrollInvoices	Delete and rebuild an entire payroll month, payments included
RecordInstallmentPaymentAccounting GeneralLedgerService	Delete-and-repost within the posting paths themselves

5,065 of the ledger's 5,084 transactions are posted (`IS_POSTED = 'Y'`). Essentially the whole ledger is reachable this way. A posted journal entry is the audit trail; deleting one destroys the evidence that it ever existed. OFBiz's own model — which this system inherits — never deletes one.

One handler already gets this right. `DeleteParty` checks for `AcctgTrans` and `AcctgTransEntry` rows and *refuses* the deletion — لا يمكن حذف هذا الطرف لوجود قيود محاسبية مسجلة عليه. The instinct exists in the codebase; it simply was never applied to the reset and delete paths. It is the model for the rest.

The audit log does not cover this. `ENTITY_AUDIT_LOG` exists and holds 1,579 rows, but its earliest entry is **31 August 2026** — six days *after* the INV1038 edit. The change that prompted this entire investigation left no audit record at all. It was reconstructed from arithmetic and a hand-written payment comment.

6.2 What professional practice requires

Four different operations are currently served by one button. They are not interchangeable:

Operation	Precondition	What should happen
Edit	Nothing posted yet	Change freely — no ledger impact

Operation	Precondition	What should happen
Reverse	Posted, period open	New contra transaction with opposite signs, dated in the open period. The original stays. Both are visible.
Void / cancel	Posted, should never have existed	Status → <i>*_CANCELLED</i> <i>plus</i> a reversal. The row is never removed.
Delete	Never posted, nothing references it	The only case where physical deletion is defensible

The statuses this needs are already seeded and unused by the reset paths: `INVOICE_CANCELLED` , `PAYMENT_CANCELLED` , `PMNT_VOIDED` , `ADVANCE_CANCELLED` and their siblings.

6.3 One qualification on the auditor's instruction

“Deletes should be marked as deleted, not removed” is correct **for financial artifacts**. Applied literally to the whole system it becomes a liability — every list query needs a filter, unique constraints break, and mistyped leads and duplicate addresses accumulate permanently.

The boundary worth agreeing in writing: **anything that has ever touched the general ledger, plus anything referenced by something that has**. Draft records that never posted can still be deleted outright.

7 Period close

A period close is the moment a stretch of time is declared finished: the month's numbers are final, and corrections found later are booked in the current open period instead of rewriting history. It is the control that would have prevented this incident outright — June would have been closed in mid-July, and the 25 August edit refused.

7.1 The system already implements it

`GeneralLedgerService.CloseFinancialTimePeriod` is a faithful port of the OFBiz routine. On close it:

Step	
1	Refuses if any child period is still open — a quarter cannot close while its months are open
2	Finds the last closed date, to know where this period starts accumulating
3	Sums all revenue, expense and income activity since then
4	Posts a <code>PERIOD_CLOSING</code> transaction — debit 850000 (profit & loss), credit 336000 (retained earnings) — rolling the period result into equity
5	Writes a <code>GL_ACCOUNT_HISTORY</code> row per GL account: opening balance, period movement, closing balance
6	Sets <code>IS_CLOSED = 'Y'</code>

Both required accounts are configured for `Company`. The machinery is complete.

7.2 It has never been used

Check	Finding
Closed periods	0 — all six rows are <code>IS_CLOSED = 'N'</code>
<code>GL_ACCOUNT_HISTORY</code> rows	0
<code>PERIOD_CLOSING</code> transactions	0
Periods defined	2 fiscal years (2025, 2026) and 4 quarters. No monthly periods at all — and payroll is monthly.

7.3 Why closing alone would not be enough

Closing a period today would be decoration. The only period-close check in the codebase is in `GeneralLedgerService.PostAcctgTrans`, and it (a) merely appends a warning message rather than blocking, and (b) is never called by the paths that create transactions — `CreateAcctgTransAndEntries` stamps `IsPosted = 'Y'` directly at creation.

So `ResetInvoice`, `ResetPayment` and the eleven others would happily delete transactions out of a closed June and nothing would stop them. Close only becomes real when it is paired with a guard on the write paths. Turning it on without that would present a control that does not control anything — worse than not having it.

7.4 Why it matters beyond this incident

- **Reported figures become reproducible.** Today a trial balance for June can return different numbers in September, with nothing recording that they changed. The `GL_ACCOUNT_HISTORY` snapshot is the permanent record of what was signed off.
- **Reports get a starting point.** A balance becomes *opening balance + period movement* rather than a re-sum of all history. At 5,084 transactions that is tolerable; it will not stay that way.

- **The P&L actually resets.** Without `PERIOD_CLOSING` entries, revenue and expense accumulate indefinitely and the period result never reaches equity.
- **It answers the standard audit question.** “Can the client change last year’s figures?” Today: yes, silently, with one button. After close: no.

8 The editing policy

The user's position is that after running payroll he should be able to correct an invoice and adjust one of the two payments to match. The requirement is legitimate — post-run corrections are normal payroll. What is unsafe is the mechanism: a hand-typed payment amount alongside a system-derived one gives two sources of truth and no way to tell which is stale.

There are **two gates**, not one, and they control different things:

State	What is allowed
Generated, not sent period open	Edit invoices freely. The two payment amounts recompute themselves and are never typed by hand. This is the flexibility being asked for, and it is safe.
Sent period still open	The run is settled — do not reopen it. A correction can still be <i>dated in that same month</i> , as a reversal or an ad-hoc payment to the individual employee.
Period closed	Nothing in that month changes. Corrections go forward — next month’s payroll, or a dated entry in the open period.

So “if it is already sent, fix it next month” is slightly stricter than necessary. The accurate rule is “**in the next open period**”, which with a mid-month close cadence often still means the same month.

One open decision

Payroll invoices are posted to the GL at *generation*, not at payment — the batch sets each to `INVOICE_READY` , which posts the accrual immediately. So even the pre-send editing window touches posted transactions. Two defensible policies:

- **Pragmatic** — within an open period, allow unpost → edit → repost on the invoice accrual. Matches what the code does today and what most mid-market ERPs permit.
- **Strict** — reverse and re-post even there. Cleaner for the auditor, more entries, more work.

Recommendation: pragmatic before send, strict after. Worth agreeing with the auditor in writing so it is not reopened later.

Suggested wording for the user. Yes — you can correct an invoice after running payroll. Do it *before* sending the two payments and their amounts will update themselves; you will not type them again. Once the payments are sent the month is settled, and a correction goes on the next run or as a separate payment to that employee. Separately: if someone should be paid by bank instead of cash, change it on the employee *before* running the month — changing it afterwards does not move them between the two payments, it only stops the existing payment from adding up.

9 Way forward

Ordering matters more than usual: periods cannot be closed over data known to be wrong, and closing without enforcement is worse than not closing.

Phase	Work	Why in this order
Now	Run <code>fix_june_payroll_inv1038_2026_09_06.sql</code> , create party 155's payment, send both	June's bank salaries are still unpaid and one employee's 60,000 has sat in his accrued account since June. Everything else depends on the data being correct first.
Next	Recompute payment amounts on invoice edit while <code>PMNT_NOT_PAID</code> ; make the amount read-only in the UI; refuse the revert-to- <code>IN_PROCESS</code> path once the payment is sent	The change that actually answers the user's request. Grants the flexibility he wants and removes the hand-typed amount that caused the incident. Highest value per unit of effort, and the first payroll-shaped instance of the general rule below.
Then	Classify all 13 delete sites: never-posted-only, needs reversal, or needs soft-delete	Cheap, and it sizes the rest of the work honestly before committing to it.
Then	Build two primitives: a <code>ReverseAcctgTrans</code> service producing linked contra entries, and a period guard called from the transaction-creation paths that <i>throws</i> rather than warns — applied to deletes and resets, not only inserts	Without these, neither soft delete nor period close means anything.
Then	Change what Reset means: unposted keeps today's behaviour, posted reverses. Soft-delete for ledger-touching artifacts via <code>*_CANCELLED</code> .	This is the auditor's finding closed out.
Then	Create monthly <code>CustomTimePeriod</code> rows as children of the quarters; begin closing on a cadence	Last, because it must follow the corrections and the guard. Consider a two-stage close — soft monthly (elevated role can still edit, logged), hard quarterly — so the discipline arrives gradually rather than as a wall.
After	Payroll correctness cleanups: single deduction engine (issue 2), valid status constants with a data fix (3), seed the three item types (4), preserve absence days in the editor (6)	Issue 2 can mark an advance repaid that was never deducted — a real money defect, not tidiness.
Later	Move attendance derivation server-side (issue 5); retire dead code and literals (7, 9, 10)	Auditability rather than correctness. Before the next audit cycle, not before the next payroll run.

The shape of the fix is the same throughout: stop deriving historical facts from mutable master data, and stop destroying evidence to correct a mistake. Snapshot what a run was executed against, record what a payment covers, reverse rather than delete, and close the months so the past stops moving.

10 How these figures were verified

Every number here was read from the `erp_contracts` database, not inferred from code. The June reconstruction rests on three independent confirmations:

- **262,078 + 60,000 = 322,078** — the payment's own comment records exactly this composition, written by hand at the time.
- **262,078 + 667 = 262,745** — the current bank run total, after the absence removal.
- **Cash payment 18786 debited 30 employees, not 31** — party 155 is absent from it, confirming he was in the bank group when the run posted, and his June accrual of 60,000 in GL **220008** remains uncleared.

The correction script was executed against a copy of the database inside a transaction and rolled back. It reported 1, 2 and 1 rows affected, an INV1038 net of 9,333.00, a balanced accrual, and — the decisive check — a bank run total of **262,078.00 across 19 invoices, equal to the corrected payment**. Re-applying it a second time changed nothing, confirming the guards.

Per-invoice totals were computed the way the application computes them — signed by **IS_POSITIVE_AMOUNT**, **quantity** × **amount**, rounded per invoice before summing — which is the rounding rule payroll payments depend on.

```
SELECT p.PREFERRED_PAYROLL_PMT_METHOD method, COUNT(*) n, ROUND(SUM(t.total),2) run_total
FROM INVOICE i JOIN PARTY p ON p.PARTY_ID = i.PARTY_ID_FROM
JOIN (SELECT ii.INVOICE_ID,
        ROUND(SUM(ii.QUANTITY*ii.AMOUNT*CASE WHEN iit.IS_POSITIVE_AMOUNT=0 THEN -1 ELSE 1 END),2) total
      FROM INVOICE_ITEM ii JOIN INVOICE_ITEM_TYPE iit USING(INVOICE_ITEM_TYPE_ID)
      GROUP BY ii.INVOICE_ID) t ON t.INVOICE_ID = i.INVOICE_ID
WHERE i.INVOICE_TYPE_ID='PAYROL_INVOICE' AND i.PARTY_ID='Company'
      AND i.INVOICE_DATE BETWEEN '2026-06-01' AND '2026-06-30'
GROUP BY 1;
```

Prepared 6 September 2026 against master @ b6035a96. Section 3 reflects code committed in 130429d1. Section 4 reflects **API/Sql/Production/fix_june_payroll_inv1038_2026_09_06.sql**, written and dry-run but not yet applied. Sections 5 to 9 are findings and recommendations, not committed work.